BZ205 Konu Anlatımı

Yığın ve Kuyruk

Hocanın ders videoları, slaytları ve sınav soruları temel alınmıştır

27 Ekim 2026

5 Yığın (Stack) ve Kuyruk (Queue)

5.1 Yığın – LIFO (son giren ilk çıkar)

Ekleme ve silme yalnızca **tepeden (top)** yapılır. Temel işlemler: `push(x)` (tepeye ekle), `pop()` (tepedekini çıkar ve döndür), `top()` (tepedekini silmeden gör), `isEmpty()`, `isFull()`.

WhatsApp'ta tespit edilen hata

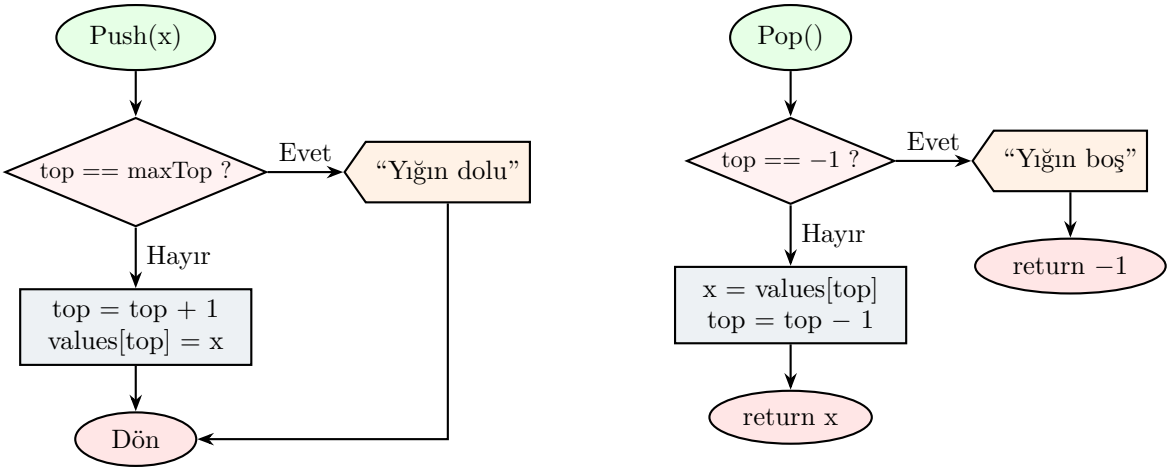
"Drive'daki farz edin ki sorularında stack.push ile if kontrolü yapılmış... Hatalı, veri alırken pop kullanıyoruz." Yığından eleman *almak* için `pop` kullanılır; yığının elemanlarına indisle (`stack[i]`) erişilmez.

Dizi ile gerçekleştirme

```
class Stack {
public:
    Stack(int size = 10) { maxTop = size - 1; values = new double[size]; top = -1; }
    bool IsEmpty() { return top == -1; }        // bos yigin: top = -1
    bool IsFull()  { return top == maxTop; }
    double Push(const double x) {
        if (IsFull()) { cout << "Yigin dolu"; return -1; }
        values[++top] = x;                      // once top artar, sonra yazilir
        return x;
    }
    double Pop() {
        if (IsEmpty()) { cout << "Yigin bos"; return -1; }
        return values[top--];                   // once okunur, sonra top azalir
    }
    double Top() { if (IsEmpty()) return -1; return values[top]; }
private:
    int maxTop;  int top;  double* values;
};
```

Varsayılan boyut (10) kullanıcının boyut vermeyi unutmasına karşı konur. C'de dizinin 0. hücresinde veri olduğu için boş yığın -1 ile gösterilir.

Akış diyagramı: Push ve Pop (dizi ile)



Boş yığında `top = -1`, dolu yığında `top == maxTop`.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Bağlantılı liste ile gerçekleştirme

`push` = listenin *başına* ekleme (`InsertNode(0,x)`), `pop` = baştaki düğümü silme. Boyut sınırı yoktur. Hoca: *"baştan ekleme silmenin zaman karmaşıklığı sabit, order of C"* [video 10 Liste 2 47:34].

```
class Stack : public List {
public:
    double Top()  { if (head == NULL) return -1; return head->data; }
    void   Push(const double x) { InsertNode(0, x); }
    double Pop() {
        if (head == NULL) { cout << "Yigin bos"; return -1; }
        double val = head->data;
        DeleteNode(val);                          // bastaki dugum silinir
        return val;
    }
};
```

Karmaşıklık: push, pop, top: $O(1)$.

5.2 Postfix (sonek) ifadenin yığınla hesaplanması

Soldan sağa okunur: **sayı** gelince `push`; **işlem** gelince iki kez `pop` – *ilk çıkan sağ işlenen*, ikinci çıkan sol işlenen – sonuç `push`. Sonda yığında kalan değer sonuçtur.

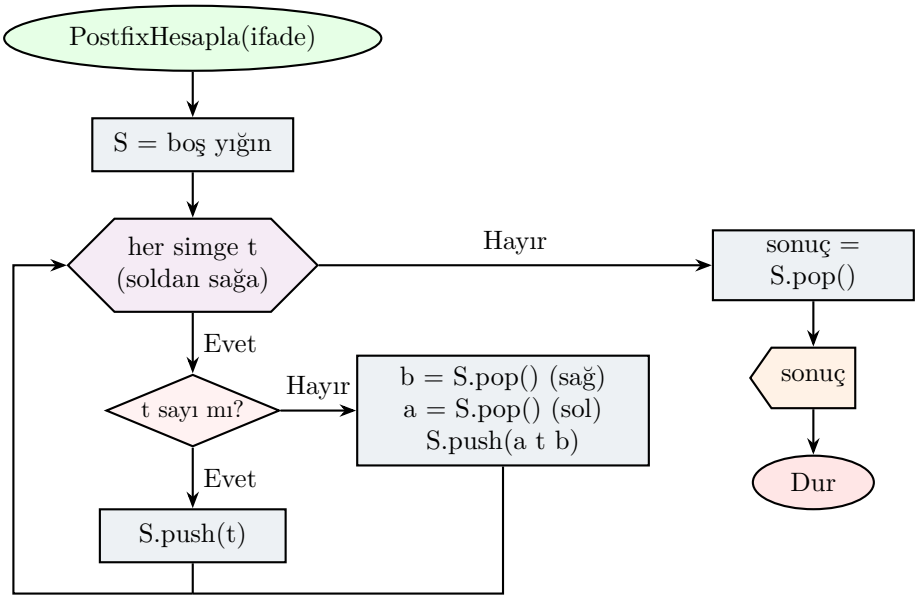
Okunan	İşlem	Yığın (alt→tepe)
5, 6, 8	push	5, 6, 8
*	$6 \times 8 = 48$	5, 48
+	$5 + 48 = 53$	53
7, 6, 2	push	53, 7, 6, 2
/	$6/2 = 3$	53, 7, 3
−	$7 - 3 = 4$	53, 4
3	push	53, 4, 3
*	$4 \times 3 = 12$	53, 12
−	$53 - 12 = 41$	**41**

(Hocanın örnek sınav sorusu: `5 6 8 * + 7 6 2 / - 3 * -` = 41.) Bölme ve çıkarmada işlenen sırası önemlidir: *"7/5 mi yapacaksın yoksa 5/7 mi?"*

```
PostfixHesapla(ifade)
  S = bos yigin
  for (her simge t, soldan saga)
     if (t sayi) S.push(t)
     else
        b = S.pop()     // sag
        a = S.pop()     // sol
        S.push(a t b)
  Yaz(S.pop())
```

2021 vizesi ve örnek sınav sorularında soruldu. Çıkarma ve bölmede ilk çıkan *sağ* işlenendir. Her simge bir kez işlenir: $\Theta(n)$.

Akış diyagramı: sonek (postfix) ifadenin yığınla hesaplanması



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

5.3 Kuyruk – FIFO (ilk giren ilk çıkar)

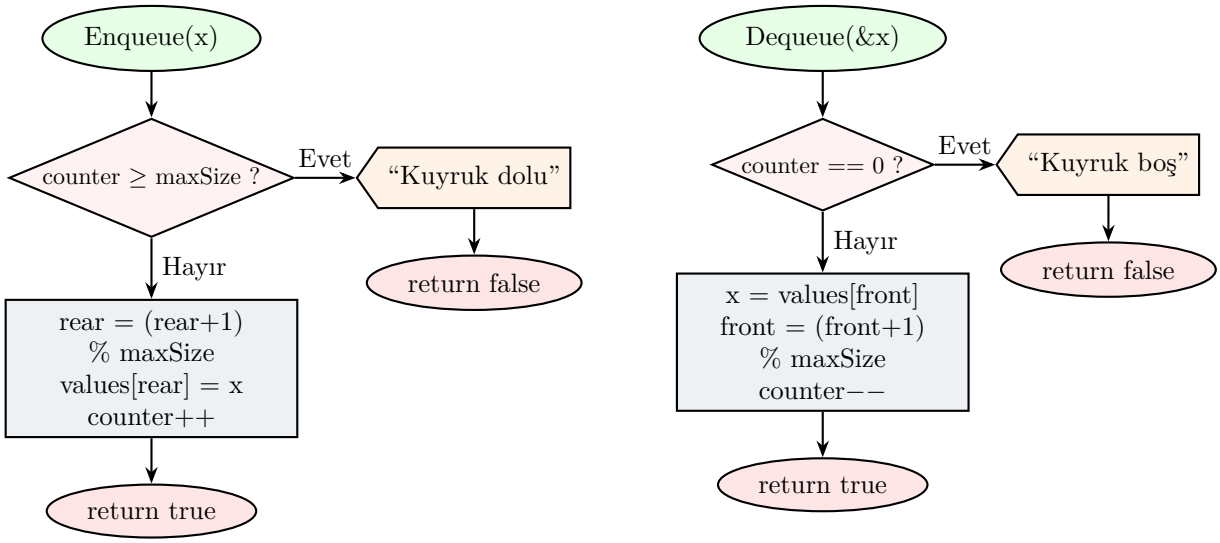
Ekleme **arkadan (rear/back)**, silme **önden (front)**. Temel işlemler: `enqueue(x)`, `dequeue()`, `isEmpty()`, `isFull()`, (`size()`).

Dizi ile dairesel kuyruk

```
class Queue {
public:
    Queue(int size = 10) { values = new double[size]; maxSize = size;
                           front = 0; rear = -1; counter = 0; }
    bool IsEmpty() { return counter == 0; }
    bool IsFull()  { return counter >= maxSize; }
    bool Enqueue(double x) {
        if (IsFull()) { cout << "Kuyruk dolu"; return false; }
        rear = (rear + 1) % maxSize;            // dairesel ilerleme
        values[rear] = x;  counter++;
```

```
        return true;
    }
    bool Dequeue(double& x) {
        if (IsEmpty()) { cout << "Kuyruk bos"; return false; }
        x = values[front];
        front = (front + 1) % maxSize;
        counter--;
        return true;
    }
private:
    int front, rear, counter, maxSize;  double* values;
};
```

Akış diyagramı: Enqueue ve Dequeue (dizi ile dairesel)



`counter` eleman sayısını tutar; `% maxSize` ile indis başa döner.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Dairesel kuyrukta enqueue ve dequeue $O(1)$. Dairesel olmayan, silmede tüm elemanları öne kaydıran dizi kuyruğunda silme $n-1$ kaydırma $\Rightarrow O(n)$ [video 11 36:04]. Bağlantılı listeyle: enqueue sona (son düğüm işaretçisi tutularak), dequeue baştan, ikisi de $O(1)$.

5.4 Farz edin ki sorularında yığın ve kuyruğu bozmadan gezmek

İki temel kalıp – Not2 ("sınıf listeleri değişmeyecek") için şart

Yığın: geçici yığına aktar, işini yap, geri aktar (iki kez ters çevrildiği için sıra korunur):

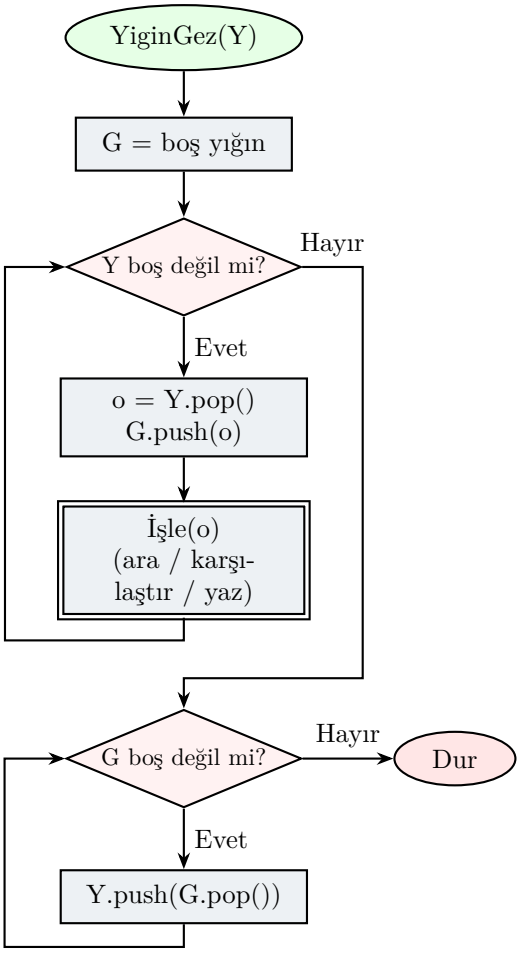
```
Stack gec
while (!Y.isEmpty())  { o = Y.pop();  gec.push(o);  ...islem... }
while (!gec.isEmpty())  Y.push(gec.pop())
```

Kuyruk: boyutu kadar döndür (çıkar–işle–arkaya geri koy); tur bitince aynı sıra:

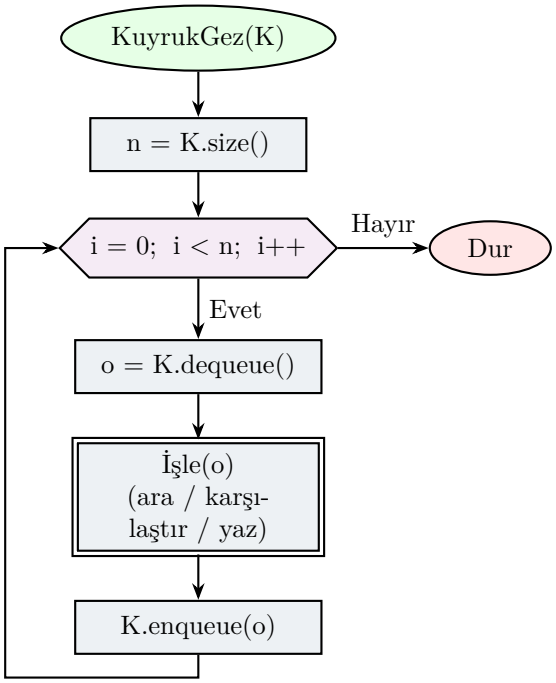
```
n = K.size()
for (i = 0; i < n; i++)  { o = K.dequeue();  ...islem...;  K.enqueue(o); }
```

Hoca derste ayrıca `yigin.clone()`, `kuyruk.clone()` ile kopya üzerinde çalıştı. Kopya fonksiyonu kullanacaksanız ne yaptığını açıklayın; geçici yapı ile geri yükleme her durumda kabul edilir.

Akış diyagramı: yığını bozmadan gezme



Akış diyagramı: kuyruğu döndürerek gezme



5.5 Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
`push` / `pop` / `top`	$\Omega(1)$	$O(1)$	$O(1)$	dizi ve bağlantılı liste gerçekleştirmesinde de sabit
`enqueue` / `dequeue` (dairesel)	$\Omega(1)$	$O(1)$	$O(1)$	`% maxSize` ile indis başa döner
`dequeue` (kaydırmalı dizi)	$\Omega(1)$	$O(n)$	$O(1)$	silmede $n-1$ eleman öne kayar
Postfix hesaplama	$\Theta(n)$	$\Theta(n)$	$O(n)$	her simge bir kez; yığında en fazla n sayı
Bozmadan gezme (yığın/kuyruk)	$\Theta(n)$	$\Theta(n)$	$O(n)$ / $O(1)$	yığında geçici yığın $O(n)$; kuyrukta döndürme ek alan istemez